

# Documento de Arquitectura de Software (SAD)

## Sistema de Gestión de Pedidos Multicanal & Control de Inventario

Proyecto: Leofit Solutions

### 1. Identificación y Control del Documento

| Campo                  | Detalle                                                                                   |
|------------------------|-------------------------------------------------------------------------------------------|
| Institución:           | Universidad Tecnológica del Perú (UTP)                                                    |
| Facultad:              | Facultad de Ingeniería de Sistemas e Informática                                          |
| Curso:                 | Integrador II: Software                                                                   |
| Proyecto:              | Sistema Web Progresivo (PWA) de Gestión de Pedidos & Control de Inventario en Tiempo Real |
| Empresa Beneficiaria:  | Leofit (PYME textil deportiva - confección y comercialización)                            |
| Dueño de la Empresa:   | Víctor Leandro Cárdenas Fernández                                                         |
| Versión del Documento: | 1.1.0                                                                                     |
| Fecha:                 | Septiembre 2026                                                                           |

#### Equipo de Desarrollo (Grupo 01):

- 1. Cárdenas Fernández Víctor Leandro — Back-End / Base de Datos
- 2. Dávila Morales Jim Alessandro — Calidad / DevOps / Despliegue
- 3. Roman Delgado Harley Anthony — UX / Front-End
- 4. Loayza Rodriguez Lady Luz — UX / Front-End / Coordinación General
- 5. Rojas Sanchez Daniel Enrique — Gestión / Análisis Funcional

### 2. Introducción y Objetivos Arquitectónicos

#### 2.1. Propósito

El presente documento describe la arquitectura técnica integral, los patrones de diseño, las vistas estructurales (modelo 4+1) y las decisiones de ingeniería (ADR) adoptadas para la construcción del sistema de información de **Leofit**.

## 2.2. Contexto del Negocio y Problemática

Leofit gestionaba la recepción de pedidos a través de múltiples canales desarticulados (WhatsApp personal, llamadas telefónicas, mensajes directos de Instagram) registrando la información en cuadernos físicos o notas sueltas. Esta operativa generaba: \* Sobreventa de prendas sin stock real disponible en el taller o almacén. \* Retrasos y confusiones en el empaque y despacho de pedidos. \* Cero trazabilidad del estado del pedido (Recibido -> Preparación -> En Camino -> Entregado). \* Falta de métricas consolidadas sobre el flujo de caja e ingresos diarios.

## 2.3. Objetivos Arquitectónicos

1. **Desacoplamiento Estricto:** Separación total entre la capa de interfaz de usuario (Frontend PWA) y la lógica de negocio/persistencia (API REST + Base de Datos).
2. **Alta Eficiencia & Baja Latencia:** Respuestas de consulta en menos de 500 ms y compilación ultrarrápida del cliente.
3. **Portabilidad y Soporte Mobile-First:** Experiencia ergonómica optimizada para smartphones con capacidad de instalación como Progressive Web App (PWA).
4. **Atomicidad e Integridad de Inventario:** Garantía ACID estricta en la reserva y descuento de prendas por talla y color.
5. **Seguridad y Trazabilidad:** Control de acceso basado en roles (RBAC) y auditoría de cambios de estado.

---

## 3. Atributos de Calidad (Requerimientos No Funcionales)

mindmap

```
root((Atributos de Calidad))
  Rendimiento
    Latencia API <= 500ms
    Carga inicial PWA < 1.5s
    Build ultrarrápido con Vite
  Seguridad
    Cifrado de contraseñas Bcrypt
    Autenticación Stateless JWT
    Políticas CORS y Sanitización
  Disponibilidad
    99% Uptime en horario comercial
    Despliegue distribuido en CDN
    Estrategia Offline con PWA
  Usabilidad
    Metodología Mobile-First
    Accesibilidad WCAG AA
    Control de Roles RBAC
  Mantenibilidad
    TypeScript estricto
    Pruebas unitarias con Vitest
    Arquitectura en capas
```

| Atributo               | Requerimiento / Métrica                                                  | Estrategia de Implementación                                                                                             |
|------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Rendimiento</b>     | Tiempo de carga inicial < 1.5s; tiempo de respuesta API <= 500 ms.       | Compilación con Vite, bundle modular, minificación con Rollup, índices en base de datos.                                 |
| <b>Seguridad</b>       | Autenticación stateless y cifrado robusto de credenciales.               | Tokens JWT firmados, contraseñas hasheadas con bcrypt (10 rounds), validación de inputs con schemas, CORS estricto.      |
| <b>Disponibilidad</b>  | 99% de disponibilidad en horario comercial (08:00 - 22:00).              | Despliegue en infraestructuras PaaS/Cloud de alta disponibilidad (GitHub Pages / Vercel + Render/Railway).               |
| <b>Escalabilidad</b>   | Capacidad de soportar picos de venta de temporada (campañas deportivas). | Arquitectura stateless en API REST que permite replicación horizontal de contenedores.                                   |
| <b>Mantenibilidad</b>  | Código limpio, tipado estático y pruebas automatizadas.                  | TypeScript estricto en frontend y backend, arquitectura en capas, pruebas con Vitest y CI/CD en GitHub Actions.          |
| <b>Usabilidad (UX)</b> | Curva de aprendizaje < 15 minutos para operadores nuevos.                | Enfoque mobile-first, diseño limpio con TailwindCSS, contraste accesible (WCAG AA) y retroalimentación visual inmediata. |

## 4. Vistas Arquitectónicas (Modelo 4+1 de Kruchten)

flowchart TD

```

    subgraph Escenarios ["Vista de Escenarios (+1)"]
        CU["Casos de Uso Críticos\n(Registrar Pedido, Control de Stock, Auditoría)"]
    end
  
```

```

    subgraph Arquitectura ["Vistas de Ingeniería"]
        VL["Vista Lógica\n(Arquitectura Multicapa & Componentes)"]
        VP["Vista de Procesos\n(Transacciones ACID & Máquina de Estados)"]
        VD["Vista de Desarrollo\n(Monorepo Modular & CI/CD)"]
    end
  
```

```

    VF["Vista Física\n(Infraestructura Cloud & Despliegue)"]
end

CU --> VL
CU --> VP
CU --> VD
CU --> VF

```

---

## 4.1. Vista Lógica: Arquitectura Multicapa y Flujo de Información

flowchart TB

```

    subgraph Presentacion ["1. Capa de Presentación (Frontend PWA)"]
        UI_Login["Módulo Login & Acceso"]
        UI_Dash["Dashboard & KPIs"]
        UI_Orders["Gestor de Pedidos"]
        UI_Products["Control de Inventario"]
        AppContext["Estado Global (AppContext / Cache)"]

        UI_Login --> AppContext
        UI_Dash --> AppContext
        UI_Orders --> AppContext
        UI_Products --> AppContext
    end

    subgraph Aplicacion ["2. Capa de Aplicación (API REST Node.js / Express)"]
        Router["Enrutador Principal & CORS"]
        AuthMiddleware["Middleware JWT & RBAC Guard"]
        Validator["Validador de Schemas (Zod)"]

        subgraph Servicios ["Servicios de Negocio"]
            AuthSvc["AuthService"]
            OrderSvc["OrderService"]
            StockSvc["InventoryEngine"]
            ReportSvc["KPIService"]
        end

        Router --> AuthMiddleware --> Validator
        Validator --> AuthSvc
        Validator --> OrderSvc
        Validator --> StockSvc
        Validator --> ReportSvc
    end

    subgraph Persistencia ["3. Capa de Acceso a Datos & Base de Datos"]
        ORM["Mapeador Objeto-Relacional (Prisma / Sequelize)"]
        DB["Base de Datos Relacional SQL\nPostgreSQL / MySQL"]

        Servicios --> ORM
    end

```

```
ORM --> DB
end
```

```
Presentacion -- "HTTPS / JSON (Bearer Token)" --> Aplicacion
```

---

## 4.2. Vista de Procesos: Secuencia Transaccional Atómica de Creación de Pedido

```
sequenceDiagram
    autonumber
    actor Operador as Dueño / Operador
    participant PWA as PWA Frontend (React)
    participant API as API Controller (Express)
    participant Service as OrderService
    participant DB as Base de Datos (SQL)

    Operador->>PWA: Ingresa datos del cliente y prendas
    PWA->>PWA: Valida formulario y autocalcula subtotal/delivery
    PWA->>API: POST /api/orders (Payload + JWT Token)
    API->>API: Valida autenticación y schema de datos
    API->>Service: createOrder(orderData)

    rect rgb(240, 248, 255)
        note right of Service: Inicio de Transacción ACID
        Service->>DB: BEGIN TRANSACTION
        Service->>DB: SELECT stock FROM product_variants WHERE id IN (...) FO
    R UPDATE

        alt Stock Insuficiente en alguna variante
            DB-->>Service: Existencias < Unidades solicitadas
            Service->>DB: ROLLBACK TRANSACTION
            Service-->>API: Error: Stock Insuficiente
            API-->>PWA: 409 Conflict (Detalle de prenda sin stock)
            PWA-->>Operador: Alerta visual de stock no disponible
        else Stock Disponible
            DB-->>Service: Existencias verificadas
            Service->>DB: UPDATE product_variants SET stock = stock - qty WHE
RE id = ...
            Service->>DB: INSERT INTO orders (order_number, client_id, total,
            status, ...)
            Service->>DB: INSERT INTO order_items (order_id, variant_id, qty,
            unit_price, ...)
            Service->>DB: INSERT INTO order_status_history (order_id, status,
            changed_at, ...)
            Service->>DB: COMMIT TRANSACTION
            note right of Service: Transacción Completada con Éxito
            Service-->>API: Orden creada satisfactoriamente
            API-->>PWA: 201 Created (Order Object + Ticket Formateado)
```

```

        PWA-->>Operador: Notificación de éxito y opción de compartir tick
et WhatsApp
    end
end

```

---

### 4.3. Máquina de Estados del Ciclo de Vida del Pedido

```

stateDiagram-v2
    [*] --> RECIBIDO: Registro de Pedido por WhatsApp/Llamada/Local

    RECIBIDO --> PREPARACION: Confirmación de Pago & Inicio de Empaque
    RECIBIDO --> CANCELADO: Cliente Desiste / Inconsistencia de Pago

    PREPARACION --> EN_CAMINO: Asignación a Motorizado / Courier
    PREPARACION --> CANCELADO: Incidencia Crítica en Taller

    EN_CAMINO --> ENTREGADO: Confirmación de Recepción por Cliente
    EN_CAMINO --> PREPARACION: Reintento de Entrega por Dirección Errónea

    ENTREGADO --> [*]: Pedido Cerrado con Éxito

    CANCELADO --> [*]: Reposición Automática de Stock en Almacén

```

---

### 4.4. Vista Física / Topología de Despliegue en Producción

```

flowchart LR
    subgraph Clientes ["Dispositivos Clientes"]
        Mobile["Smartphone Android / iOS\n(PWA Instalada)"]
        Desktop["Navegador Web Desktop\n(Panel Administrativo)"]
    end

    subgraph CDN ["Red de Distribución (Edge / CDN)"]
        GHPages["GitHub Pages / Vercel CDN\n(HTML5, CSS3, JS Bundles)"]
    end

    subgraph CloudApp ["Capa de Aplicación Cloud (PaaS)"]
        ReverseProxy["Reverse Proxy / SSL TLS 1.3\n(Cloudflare / Nginx)"]
        NodeRuntime["Node.js 22 LTS Runtime\n(Express REST API Server)"]
        ReverseProxy --> NodeRuntime
    end

    subgraph CloudDB ["Persistencia Gestionada"]
        SQLDB["Managed PostgreSQL / MySQL\nPool de Conexiones + Backups Diari
os)"]
    end
end

```

```
Mobile -- "HTTPS" --> GHPages
Desktop -- "HTTPS" --> GHPages
Mobile -- "REST API (JSON / HTTPS)" --> ReverseProxy
Desktop -- "REST API (JSON / HTTPS)" --> ReverseProxy
NodeRuntime -- "Encrypted TCP / SSL" --> SQLDB
```

---

## 4.5. Vista de Datos: Modelo Entidad-Relación (ERD)

erDiagram

```
USERS ||--o{ ORDERS : "registra / audita"
CLIENTS ||--o{ ORDERS : "realiza"
CATEGORIES ||--|{ PRODUCTS : "clasifica"
PRODUCTS ||--|{ PRODUCT_VARIANTS : "posee"
PRODUCT_VARIANTS ||--o{ ORDER_ITEMS : "incluido_en"
ORDERS ||--|{ ORDER_ITEMS : "contiene"
ORDERS ||--|{ ORDER_STATUS_HISTORY : "registra_historial"
```

```
USERS {
    int id PK
    string name
    string email UK
    string password_hash
    enum role "ADMIN, OPERATOR"
    datetime created_at
}
```

```
CATEGORIES {
    int id PK
    string name
    text description
}
```

```
PRODUCTS {
    int id PK
    int category_id FK
    string name
    text description
    decimal base_price
    string image_url
    boolean is_active
}
```

```
PRODUCT_VARIANTS {
    int id PK
    int product_id FK
    string size "S, M, L, XL"
    string color
    string sku UK
}
```

```

        int stock
        int alert_threshold
    }

    CLIENTS {
        int id PK
        string full_name
        string phone
        text address
        string district
        text reference
    }

    ORDERS {
        int id PK
        string order_number UK
        int client_id FK
        enum status "RECIBIDO, PREPARACION, EN_CAMINO, ENTREGADO, CANCELADO"
        decimal subtotal
        decimal shipping_cost
        decimal total_amount
        enum payment_method "YAPE, PLIN, TRANSFERENCIA, CONTRAENTREGA"
        text notes
        datetime created_at
        datetime updated_at
    }

    ORDER_ITEMS {
        int id PK
        int order_id FK
        int variant_id FK
        int quantity
        decimal unit_price
        decimal subtotal
    }

    ORDER_STATUS_HISTORY {
        int id PK
        int order_id FK
        string previous_status
        string new_status
        datetime changed_at
        text comments
    }

```

---



## 5. Registro de Decisiones de Arquitectura (ADR)

### ADR-01: Adopción de Arquitectura Desacoplada (PWA + API REST)

- **Estado:** Aceptado.
- **Contexto:** Se requería una solución que permita atender pedidos desde el celular en el taller y desde computadoras en el punto de venta.
- **Decisión:** Implementar una Progressive Web App (PWA) construida con React 19 + TypeScript, consumiendo una API REST desacoplada.
- **Consecuencias Positivas:** Independencia de despliegue, experiencia nativa en móviles sin coste de tiendas de aplicaciones, reutilización de la API para futuras integraciones.

### ADR-02: Selección de Base de Datos Relacional (SQL)

- **Estado:** Aceptado.
- **Contexto:** Las transacciones de venta e inventario requieren estricta consistencia para no permitir sobreventas.
- **Decisión:** Utilizar un motor relacional (PostgreSQL / MySQL) con soporte de claves foráneas e índices B-Tree.
- **Consecuencias Positivas:** Cumplimiento total de propiedades ACID; integridad referencial garantizada; facilidad para reportes contables.

### ADR-03: Autenticación Stateless con JSON Web Tokens (JWT)

- **Estado:** Aceptado.
- **Contexto:** Se necesita un esquema de autenticación seguro, ligero y que no dependa de sesiones en memoria en el servidor.
- **Decisión:** Emisión de tokens JWT firmados con algoritmo HMAC-SHA256, con payload que incluye userId y role.
- **Consecuencias Positivas:** Escalabilidad horizontal sin replicación de sesiones; consumo seguro desde navegadores y dispositivos móviles.

### ADR-04: Enfoque de Pruebas Automatizadas con Vitest

- **Estado:** Aceptado.
- **Contexto:** Validación rápida de las reglas de cálculo (subtotales, descuentos, cálculo de riesgo de pedidos) en el flujo de CI.
- **Decisión:** Utilizar Vitest integrado de forma nativa con Vite.
- **Consecuencias Positivas:** Ejecución de pruebas unitarias en menos de 200 ms sin sobrecarga de configuración de Babel/Jest.

---

## 6. Estrategia de Seguridad y Gobernanza

### 1. Gestión de Variables de Entorno:

- Cero secretos en código fuente. Parámetros sensibles (JWT\_SECRET, DB\_URL, credenciales) inyectados mediante archivos .env excluidos del control de versiones.

## 2. **Control de Acceso Basado en Roles (RBAC):**

- **Rol Administrador (Dueño):** Acceso total a creación de pedidos, edición de precios, control de inventario y visualización de montos de facturación.
- **Rol Operador:** Acceso restringido a cambio de estados operativos y visualización de datos de entrega, con ocultamiento de métricas financieras.

## 3. **Pipeline de Integración y Despliegue Continuo (CI/CD):**

- Validación automática de linting, tipado estático, suite de pruebas unitarias y escaneo de vulnerabilidades (security-scan.yml) ante cada Pull Request hacia la rama main.